

Requisitos – Clínica Vida +

-Requisitos Funcionais (Atendimentos):

♥ [RF001] Listagem geral de consultas

O sistema deve permitir listar todas as consultas cadastradas no banco de dados.

Atores: ADM.

Prioridade: Essencial.

- **Entradas e Pré-Condições:** O usuário acessa a rota correspondente; o banco de dados deve estar ativo.
- **Saídas e Pós-Condições:** O sistema retorna uma lista contendo todas as consultas existentes.

♥ [RF002] Consultas por especialidade

O sistema deve permitir consultar a quantidade de atendimentos agrupados por especialidade (cargo do funcionário).

Atores: Todos os Funcionários.

Prioridade: Essencial.

- **Entradas e Pré-Condições:** Existência de funcionários e consultas cadastradas.
- **Saídas e Pós-Condições:** O sistema retorna um agrupamento de especialidades com a quantidade de consultas para cada.

♥ [RF003] Quantidade de pacientes por médico

O sistema deve permitir consultar quantos pacientes distintos cada médico atendeu.

Atores: Todos os Funcionários.

Prioridade: Essencial.

- **Entradas e Pré-Condições:** O sistema deve possuir consultas registradas com vínculo entre paciente e médico.
- **Saídas e Pós-Condições:** O sistema retorna cada médico com a quantidade de pacientes únicos atendidos.

♥ [RF004] Listagem de consultas com dados completos

O sistema deve exibir consultas acompanhadas dos dados do paciente e especialidade do médico.

Atores: Todos os Funcionários.

Prioridade: Essencial.

- **Entradas e Pré-Condições:** Consultas, pacientes e funcionários devem existir e estar corretamente relacionados.
- **Saídas e Pós-Condições:** O sistema retorna os dados combinados (paciente, especialidade, data e status).

♥ [RF005] Listar pacientes atendidos pelo médico logado

O sistema deve permitir ao médico visualizar todos os seus pacientes vinculados a consultas cadastradas.

Atores: Médicos.

Prioridade: Essencial.

- **Entradas e Pré-Condições:** O médico deve estar logado e o header user deve conter o ID válido.
- **Saídas e Pós-Condições:** O sistema retorna uma lista contendo somente os pacientes deste médico.

♥ [RF006] Cadastro de consulta

O sistema deve permitir cadastrar novas consultas respeitando regras de integridade e disponibilidade.

Atores: ADM e Atendentes.

Prioridade: Essencial.

- **Entradas e Pré-Condições:**
 - Dados enviados: id_paciente, id_funcionario, data, hora e status.
 - A data da consulta deve ser igual ou posterior ao dia atual.
 - Não pode existir uma consulta duplicada para o mesmo funcionário, paciente e data.
- **Saídas e Pós-Condições:**

- Caso válido: A consulta é cadastrada e retorna seus dados.
- Caso inválido: O sistema retorna erro indicando data inválida ou duplicidade.

♥ [RF007] Exclusão de consulta

O sistema deve permitir excluir uma consulta existente.

Atores: ADM.

Prioridade: Essencial.

- **Entradas e Pré-Condições:** O usuário informa o ID da consulta a ser excluída.
- **Saídas e Pós-Condições:**
 - Consulta excluída: o sistema confirma exclusão.
 - Consulta inexistente: o sistema informa que não foi encontrada.
 - Erro de comunicação: o sistema alerta sobre falha de conexão.

-Requisitos Não Funcionais (Atendimentos):

♥ [RNF001] Conexão com o banco de dados (db)

O sistema deve manter uma conexão estável com o banco de dados MySQL por meio da configuração presente em connection.js. Além de permitir consultas rápidas para evitar lentidão no retorno das rotas.

Prioridade: Essencial.

♥ [RNF002] Segurança com uso de JWT (jsonwebtoken)

O sistema deve utilizar JSON Web Token para validação e autenticação segura de usuários, garantindo acesso apenas a rotas permitidas. Tokens devem ser válidos para evitar acessos indevidos.

Prioridade: Essencial.

♥ [RNF003] Criptografia (crypto)

O sistema deve utilizar algoritmos de criptografia fornecidos pelo módulo nativo crypto para operações que exijam maior segurança (como geração de hashes, tokens ou validações internas).

Prioridade: Importante.

♥ [RNF004] Tempo de resposta do sistema

O retorno das rotas deve ocorrer em até 3 segundos para garantir boa experiência ao usuário.

Prioridade: Essencial.

♥ [RNF005] Tratamento de erros e robustez

O sistema deve identificar falhas na execução das queries e retornar respostas claras como Erro ao buscar, Não encontrado ou falhas de conexão.

Prioridade: Importante.

♥ [RNF006] Padrões de codificação

O código deve seguir boas práticas de organização, padronização de funções assíncronas e estrutura clara de rotas e respostas HTTP.

Prioridade: Importante.

-Requisitos Funcionais (Funcionários):

♥ [RF001] Listagem de funcionários

O sistema deve permitir listar todos os funcionários cadastrados no banco de dados.

Atores: ADM.

Prioridade: Essencial.

- **Entradas e Pré-Condições:** O usuário acessa a rota de listagem; o banco deve estar ativo.
- **Saídas e Pós-Condições:** O sistema retorna a lista completa dos funcionários registrados.

♥ [RF002] Cadastro de funcionário

O sistema deve permitir cadastrar novos funcionários com nome, email, data de nascimento, cargo e senha criptografada.

Atores: ADM.

Prioridade: Essencial.

➤ Entradas e Pré-Condições:

- Dados obrigatórios enviados no corpo da requisição: nome, email, data_nascimento, cargo e senha.
- A senha deve ser criptografada antes do armazenamento.

➤ Saídas e Pós-Condições:

- Funcionário é adicionado ao banco.
- O sistema retorna os dados cadastrados, incluindo o hash da senha.

♥ [RF003] Exclusão de funcionário

O sistema deve permitir excluir um funcionário existente com base no seu ID.

Atores: ADM.

Prioridade: Essencial.

➤ Entradas e Pré-Condições:

- O ID deve ser informado via parâmetros da rota.
- O funcionário deve existir no banco.

➤ Saídas e Pós-Condições:

- Exclusão bem-sucedida: mensagem de confirmação.
- Funcionário inexistente: sistema informa que não foi encontrado.
- Falhas de conexão resultam em mensagem de erro apropriada.

♥ [RF004] Login de funcionários

O sistema deve permitir que funcionários se autentiquem usando email e senha criptografada.

Atores: Todos os Funcionários.

Prioridade: Essencial.

➤ **Entradas e Pré-Condições:**

- Email e senha devem ser enviados no corpo da requisição.
- A senha deve ser transformada em MD5 antes da validação.
- O funcionário deve existir com essas credenciais no banco.

➤ **Saídas e Pós-Condições:**

- Em caso de sucesso: o sistema retorna um token JWT válido por 60 minutos.
- Em caso de falha: retorna erro de credenciais incorretas.
- Em caso de erro interno: o sistema retorna mensagem de falha no servidor.

-Requisitos Não Funcionais (Funcionários):

♥ **[RNF001] Conexão com o banco de dados (db)**

O sistema deve utilizar uma conexão MySQL estável para realizar consultas e operações CRUD na tabela de funcionários. O retorno das queries deve ser confiável e rápido, evitando falhas na execução das rotas.

Prioridade: Essencial.

♥ **[RNF002] Criptografia de senhas (crypto)**

O sistema deve criptografar todas as senhas de funcionários utilizando hashing antes de armazená-las ou compará-las.

Prioridade: Essencial.

♥ **[RNF003] Autenticação segura com JWT (jsonwebtoken)**

O sistema deve utilizar JSON Web Token para autenticação após o login, garantindo segurança e acesso autorizado às rotas protegidas. O token deve conter dados essenciais do funcionário (id, nome, email, cargo) e expirar em 60 minutos.

Prioridade: Essencial.

♥ [RNF004] Tempo de resposta da API

As rotas devem responder em até no máximo 3 segundos, garantindo uma operação rápida e fácil na listagem, autenticação e manipulação de funcionários.

Prioridade: Essencial.

♥ [RNF005] Tratamento de erros

O sistema deve capturar e tratar erros gerados por falhas no banco, dados incorretos ou problemas internos, retornando mensagens claras e apropriadas.

Prioridade: Importante.

♥ [RNF006] Conformidade com padrões de codificação

O código deve seguir boas práticas de organização, modularização e formatação, utilizando funções assíncronas e respostas HTTP adequadas.

Prioridade: Importante.

